Dockerize vos Micorservices

Objectifs :

1. Générer les fichiers .jar pour les projets
2. Créer le Dockerfile pour les autres projets
3. Utiliser Docker Compose
4. Modifier le fichier application.yml du projet department
5. Ajouter le projet gateway server
6. Créer une deuxième instance du microservice department
7. Exercice : Créer un container pour « teacher »
8. Solution de l'exercice

Générer les fichiers .jar pour les autres projets

Microservice Department :

- Revenir à l'utilisation de la BD H2
- Réactiver l'utilisation de eureka et config server

```
datasource:
  url: 'jdbc:h2:mem:depdb'
h2:
 console:
  enabled: true
```

```
cloud:
  config:
    enabled: true
  discovery:
    enabled: true
```

- Tester, puis générer le fichier .jar

Générer le fichier .jar pour le microservice eureka

Générer le fichier .jar pour le microservice config server

Créer le Dockerfile pour les autres projets

Dockerfile du projet eureka server

```
#Start with a base image containing Java runtime
FROM eclipse-temurin:21-jdk

#Information around who maintains the image
LABEL "org.opencontainers.image.authors"="nadhem.com"
# Add the application's jar to the image
COPY target/eurekaserver-0.0.1-SNAPSHOT.jar eurekaserver-0.0.1-SNAPSHOT.jar
# execute the application
ENTRYPOINT ["java", "-jar", "eurekaserver-0.0.1-SNAPSHOT.jar"]
```

Dockerfile du projet configserver

```
#Start with a base image containing Java runtime
FROM eclipse-temurin:21-jdk

#Information around who maintains the image
LABEL "org.opencontainers.image.authors"="nadhem.com"
# Add the application's jar to the image
COPY target/configserver-0.0.1-SNAPSHOT.jar configserver-0.0.1-SNAPSHOT.jar
# execute the application
ENTRYPOINT ["java", "-jar", "configserver-0.0.1-SNAPSHOT.jar"]
```

Dockerfile du projet department

```
#Start with a base image containing Java runtime
FROM eclipse-temurin:21-jdk

#Information around who maintains the image
LABEL "org.opencontainers.image.authors"="nadhem.com"
# Add the application's jar to the image
COPY target/department-microservice-0.0.1-SNAPSHOT.jar department-micro-
service-0.0.1-SNAPSHOT.jar
# execute the application
ENTRYPOINT ["java", "-jar", "department-microservice-0.0.1-SNAPSHOT.jar"]
```

Utiliser Docker Compose

Dans un premier temps on va créer, avec Docker compose des containers pour les projets : eureka, config server et department :

Créer sur la racine du projet le fichier docker-compose.yml :

```
services:
  eurekaserver:
   build: ./eurekaserver
   container_name: eurekaserver
   ports:
      - "8761:8761"
   healthcheck:
     test:  "curl --fail --silent localhost:8761/actuator/health | grep UP || exit 1"
      interval: 10s

  configserver:
    build: ./configserver
    container_name: configserver
    ports:
      - "9999:9999"
    healthcheck:
      test: "curl --fail --silent localhost:9999/actuator/health | grep UP || exit 1"
      interval: 10s

  departments-ms:
    build: ./department-microservice
    container_name: departments-ms
    ports:
      - "8080:8080"
    environment:
      - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
      - CONFIG_SERVICE_URL=http://configserver:9999
    healthcheck:
     test: "curl --fail --silent localhost:8080/actuator/health | grep UP || exit 1"
    depends_on:
      eurekaserver :
       condition: service_healthy
      configserver:
        condition: service_healthy
```

Modifier le fichier application.yml du projet department comme suit :

```
config:
    import: optional:configserver:${CONFIG_SERVICE_URL:http://local-
host:9999}
eureka:
  client:
    service-url:
      defaultZone: ${DISCOVERY_SERVICE_URL:http://localhost:8761/eureka}
```

Lancer :

docker compose up



Ajouter le projet gateway server

- Modifier le fichier application.yml :

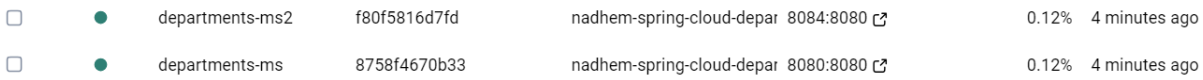
```
eureka:
  client:
    register-with-eureka: true
    fetchRegistry: true
    serviceUrl:
      defaultZone: ${DISCOVERY_SERVICE_URL:http://localhost:8761/eureka}
      instance:
          prefer-ip-address: true
```

- Générer le fichier .jar pour le projet Gateway
- Créer le fichier Dockerfile pour le projet Gateway
- Ajouter les lignes ci-dessous au fichier docker-compose.yml :

```
  gatewayserver:
  build: ./gatewayserver
  container_name: gatewayserver
  ports:
    - "8888:8888"
  environment:
    - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
  depends_on:
    eurekaserver:
      condition: service_healthy
```

Créer une deuxième instance du microservice department :

```
departments-ms2:
  build: ./department-microservice
  container_name: departments-ms2
  ports:
    - "8084:8080"
  environment:
    - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
    - CONFIG_SERVICE_URL=http://configserver:9999
  healthcheck:
    test: "curl --fail --silent localhost:8084/actuator/health | grep UP ||
exit 1"
  depends_on:
    eurekaserver:
      condition: service_healthy
    configserver:
      condition: service_healthy
```



Exercice : Créer un container pour « teacher »

Ajoutez, au fichier docker-compose.yml, le code permettant de créer un container pour le microservice teacher, qui dépend des microservices eureka, config server et department-microservice

Solution de l'exercice :

- Modifier le fichier application.yml du projet teacher comme suit :

```
spring:
 config:
    import: optional:configserver:${CONFIG_SERVICE_URL:http://lo-
calhost:9999}
eureka:
  client:
    service-url:
      defaultZone: ${DISCOVERY_SERVICE_URL:http://localhost:8761/eureka}
```

- Générer avec Maven, de nouveau le fichier .jar
- Créer le fichier Dockerfile pour le projet teacher
- docker-compose.yml :

```
teachers-ms:
  build: ./teacher
  container_name: teachers-ms
  ports:
    - "8081:8081"
  environment:
    - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
    - CONFIG_SERVICE_URL=http://configserver:9999
  depends_on:
    eurekaserver:
      condition: service_healthy
    configserver:
      condition: service_healthy
    departments-ms:
      condition: service_healthy
```

Lancer :

docker compose up